ÖVNINGSUPPGIFTER: LEKTION 4

Driftoptimering, Indexering, Exekveringsplaner och Constraints

🟢 Gröna uppgifter (Indexeringens mekanik & B-träd)

Uppgift 1

Skapa en Hög (Heap) – Skriv ett DDL-skript som skapar en tabell `dbo.AuditLogs_Heap` helt utan primärnyckel eller index. Lägg till kolumnerna `Id INT`, `LogMessage NVARCHAR(500)` och `CreatedAt DATETIME`.

Uppgift 2

Det automatiska indexet (Clustered) – Skriv ett skript som skapar tabellen `dbo.AuditLogs_Clustered` med exakt samma kolumner som i Uppgift 1, men lägg till en `PRIMARY KEY CLUSTERED` på kolumnen `Id`.

Uppgift 3

Det kirurgiska snittet (Non-Clustered) – Skapa ett vanligt, icke-klustrat index med namnet `IX_AuditLogs_CreatedAt` på kolumnen `CreatedAt` i tabellen `dbo.AuditLogs_Clustered`.

Uppgift 4

Verifiera indexstrukturen – Skriv en systemfråga mot `sys.indexes` som visar namn och `type_desc` för alla index som existerar på tabellen `dbo.AuditLogs_Clustered`.

Uppgift 5

Säkerställa unika värden – Skapa en `UNIQUE` constraint på kolumnen `ServerName` i tabellen `dbo.Servers`. Förklara i en kommentar vilket index SQL Server skapar automatiskt under motorhuven för att lösa detta.

🟡 Gula uppgifter (Exekveringsplaner & Täckande index)

Uppgift 6

Detektera en Table Scan – Slå på "Actual Execution Plan" (Ctrl + M). Skriv och kör en sökfråga som letar efter ett specifikt textmeddelande i din Heap-tabell från Uppgift 1. Vilken ikon varnar SQL Server för i planen?

Uppgift 7

Blixtsnabb sökning (Index Seek) – Kör en sökfråga mot `dbo.AuditLogs_Clustered` där du filtrerar på ett specifikt ID (`WHERE Id = 1`). Verifiera i planen att operationen har skiftat till en `Clustered Index Seek`.

Uppgift 8

Den dolda kostnaden (Key Lookup) – Kör en sökfråga där du filtrerar på `CreatedAt` men väljer (`SELECT`) kolumnen `LogMessage`. Identifiera varningssymbolen `Key Lookup` i exekveringsplanen.

Uppgift 9

Den perfekta skölden (Covering Index) – Skriv om indexet från Uppgift 3 och använd nyckelordet `INCLUDE` för att lägga till kolumnen `LogMessage` i lövnoderna. Kör om frågan från Uppgift 8 och verifiera att din `Key Lookup` är helt borta.

Uppgift 10

SARGable kodtvätt – Du har fått en långsam fråga: `SELECT * FROM dbo.Incidents WHERE YEAR(ReportedAt) = 2026;`. Skriv om denna fråga så att den blir SARGable och kan utnyttja index på kolumnen `ReportedAt`.

Röda uppgifter (Databasoptimering, Låsningar & Privilegier)

Uppgift 11

Indexera länkade nycklar (FK Optimering) – Skriv det index som *alltid* bör skapas på en Foreign Key-kolumn för att skydda raderings- och valideringsprestanda, applicerat på kolumnen `TargetServerId` i tabellen `dbo.Incidents`.

Uppgift 12

Contradiction Detection i praktiken – Lägg till en `CHECK`-constraint på tabellen `dbo.Servers` som begränsar `IsActive` till antingen 0 eller 1. Skriv sedan en fråga `WHERE IsActive = 5` och verifiera i exekveringsplanen att SQL Server inte ens rör disken.

Uppgift 13

Implicit konverteringsjakt – Om kolumnen `MacAddress` i en tabell är definierad som `VARCHAR(17)`, visa med ett T-SQL-exempel hur en inmatad unicode-sträng (NVARCHAR, t.ex. `N'00-14-22-01-23-45'`) tvingar fram en implicit konvertering och förstör indexprestanda.

Uppgift 14

Smutsiga läsningar (NOLOCK) – Skriv en sökfråga mot `dbo.Servers` som använder isoleringsnivån `READ UNCOMMITTED` (eller `NOLOCK`-hinten). Ange i en kommentar när detta är acceptabelt och när det är direkt livsfarligt ur ett finansiellt perspektiv.

Uppgift 15

Minsta behörighet vid prestandafelsökning – Skapa en databasanvändare `DB_Profiler` utan login. Tilldela denna användare *enbart* de systemrättigheter som krävs för att kunna köra `SHOWPLAN` och läsa exekveringsplaner i din databas, helt enligt principen om Least Privilege.